

Sprint Documentation

Sprint #13	Start Date: 17/03/2026	Final Date: 23/03/2026
Projetc Name:	civitas-backend	
Ano: 5º	Análise e Desenvolvimento de Sistemas - AMS	

Team Members

Gustavo Henrique Moreira Porto
Felipe Pacheco Bianchini

Sprint Backlog

Task #	Description	Start Date	Final date
052	Validações de Cadastro de Usuário	17/03/2026	23/03/2026

Task Description

Task #	Description	Assigned To	Status	Estimated Hours	Logged Hours
052	Garantir que o cadastro de usuários possua validações completas de integridade, padronização e segurança, evitando dados inconsistentes e vulnerabilidades no sistema.	Gustavo Porto e Felipe Pacheco	Completo	20 Horas	20 Horas

Sprint Description

Tarefa: Validações de Cadastro de Usuário

Objetivo

Garantir que o cadastro de usuários possua validações completas de integridade, padronização e segurança, evitando dados inconsistentes e vulnerabilidades no sistema.

Descrição

Implementar validações no backend para os campos do cadastro de usuário, assegurando:

- Dados obrigatórios
- Formatação correta
- Padronização (normalização)
- Unicidade no banco
- Segurança (senha com hash)

Critérios de Aceite

◆ Campos Obrigatórios

-
- Validar que os campos obrigatórios não estão vazios:
 - cpf
 - nome
 - rg
 - email
 - senha
 - logradouro
 - numero
 - bairro
 - cidade
 - estado
 - cep
 - matricula
 - tipoUsuario
 - situacao

◆ Normalização de Dados

- Permitir entrada de cpf, rg e cep com máscara
- Remover caracteres não numéricos antes de processar:
 - CPF
 - RG
 - CEP

Armazenar apenas números no banco

◆ Validação de CPF

- CPF deve conter 11 dígitos após normalização

CPF deve ser válido

CPF não pode estar duplicado no banco

Rejeitar CPFs inválidos

◆ **Validação de Email**

- Validar formato de email válido

Email não pode estar duplicado no banco

Limitar tamanho (máx. 255 caracteres)

◆ **Validação de Senha**

- Mínimo de 8 caracteres
- Deve conter pelo menos:
 - 1 letra
 - 1 número

Não armazenar senha em texto puro

Armazenar senha utilizando hash seguro

Validar senha apenas via comparação de hash

◆ **Validação de Nome**

- Mínimo de 3 caracteres

Máximo de 150 caracteres

◆ **Validação de Endereço**

- logradouro: obrigatório e com tamanho máximo (max: 200)

numero: obrigatório

bairro: obrigatório

cidade: obrigatório

- estado: obrigatorio
 - Deve conter uma UF válida (ex: SP, RJ, MG)
- cep:
 - Deve conter 8 dígitos após normalização

◆ **Validação de RG**

- Campo obrigatorio

Deve conter apenas numeros após normalização (Lembre-se que os RGs atuais podem vir com letra. Portanto, se atente a isso)

Limitar tamanho (max: 20 caracteres)

◆ **Validação de Matrícula**

- Deve ser única no sistema

Não pode ser vazia (campo obrigatorio)

◆ Validação de Situação

-
- Aceitar apenas valores válidos:
 - 1 = Ativo
 - 2 = Inativo

◆ Validação de Tipo de Usuário

- Deve corresponder a um valor válido do sistema (enum)

Rejeitar valores fora do padrão

Observações e Sugestões

- Podem utilizar DataAnnotations para validações básicas, mas não se prendam a isso. Utilize formas mais seguras para informações mais sensíveis.
- Utilize o BCrypt para o gerar o hash de senha.
- Tente criar códigos reutilizáveis para facilitar a manutenção futura e escalabilidade do sistema.
- Validações preferencialmente na camada SERVICE.
- Documentem as alterações/adições.
- Verifiquem o trabalho antes do envio.

Sprint Results

Durante a Sprint 13 foi realizada a implementação de validações completas no processo de cadastro e atualização de usuários, com o objetivo de garantir a integridade, padronização e segurança dos dados armazenados no sistema.

As validações foram aplicadas principalmente na camada de serviço (service), assegurando que todos os campos obrigatórios do usuário sejam devidamente preenchidos antes de qualquer persistência no banco de dados. Além disso, foi implementada a normalização de dados como CPF, RG e CEP, removendo quaisquer máscaras ou caracteres especiais e armazenando apenas os valores numéricos.

O CPF passou a ser validado seguindo as regras oficiais, incluindo a verificação de seus 11 dígitos e aplicação do algoritmo de validação, além da checagem de duplicidade para impedir registros repetidos. Da mesma forma, o email é validado quanto ao seu formato, limitado a 255 caracteres e também verificado quanto à unicidade no sistema.

A senha do usuário passou a seguir critérios mínimos de segurança, exigindo pelo menos 8 caracteres, incluindo letras e números. Para garantir a proteção dos dados, as senhas não são armazenadas em texto puro, sendo convertidas para hash utilizando o algoritmo BCrypt. No caso de atualização de usuário (método PUT), o envio da senha é opcional: caso não seja informada, o sistema mantém o hash existente; caso seja enviada, um novo hash é gerado.

Outros campos do usuário, como nome, endereço, estado (UF), RG, matrícula, situação e tipo de usuário, também passaram a ser validados conforme regras definidas pelo sistema. Além disso, por questões de segurança, a senha deixou de ser retornada nas respostas da API.

Foram definidos retornos HTTP adequados para cada tipo de situação: respostas do tipo 400 (Bad Request) são utilizadas quando há falhas de validação, enquanto respostas 409 (Conflict) são retornadas em casos de duplicidade de dados como CPF, email ou matrícula.

Do ponto de vista técnico, a classe `UsuarioService` foi modificada para sobrescrever os métodos de criação e atualização, incorporando as validações, normalizações e o processo de hash de senha. O `UsuarioRepository` passou a incluir verificações de existência para CPF, email e matrícula, evitando duplicidades. Também foram realizados ajustes no `UsuarioBuilder` para suportar novos limites e restrições, incluindo índices únicos no banco de dados.

Foi criada uma migration chamada `20260318164700_Sprint13UsuarioValidation`, responsável por aplicar as alterações estruturais necessárias no banco. Além disso, foi implementada uma abstração específica para tratamento de senha, denominada `BCryptPasswordHashService`, e o mapeamento entre entidade e DTO foi ajustado para garantir que a senha não seja exposta nas respostas.

Para assegurar a qualidade da implementação, foram desenvolvidos testes automatizados cobrindo diversos cenários, como cadastro válido com dados normalizados, validação de CPF e email inválidos, rejeição de senhas fracas, detecção de duplicidade de dados e validação de campos como estado, situação e tipo de usuário. Também foram testados cenários de atualização com e sem alteração de senha, garantindo o comportamento correto do sistema.

Durante a execução, foram realizados testes e builds do projeto utilizando comandos específicos do .NET, sendo necessário configurar a variável de ambiente `DOTNET_ROLL_FORWARD=Major` para garantir a compatibilidade com a versão net9.0.

Como observação final, foi identificado um aviso relacionado a uma dependência vulnerável na biblioteca AutoMapper (versão 15.0.1). No entanto, essa questão não fazia parte do escopo da sprint e foi registrada como uma pendência para tratamento futuro.

Assinaturas